Formalización Problema Mochila 0/1

Jacobo Arroyave Ortiz, Camilo Niño Galvez

Facultad de Inteligencia Artificial e Ingenierías Universidad de Caldas

Análisis y diseño de algoritmos

Doc. Luz Enith Guerrero Mendieta

Noviembre 05, 2026

MOCHILA 0/1

Subproyecto 3: (Parte 1) Aceleración de la Ruta Crítica bajo Presupuesto

Alcance

Esta sección formaliza la Parte 1 del Subproyecto 3: seleccionar un subconjunto de etapas de una cadena en serie para acelerarlas bajo un presupuesto limitado. La implementación resuelve el problema mediante programación dinámica exacta y está organizada en capas de dominio, aplicación, infraestructura y presentación.

La cadena es secuencial; por tanto, todas sus etapas pertenecen a la ruta crítica y la duración total es la suma de las duraciones individuales. La cobertura de auditoría corresponde a otra parte del subproyecto y no forma parte de este análisis.

---

1. Formalización del problema de decisión: CHAIN-ACCEL-DEC

1.1 Instancia de entrada

Una instancia de **CHAIN-ACCEL-DEC** está compuesta por:

- Un entero positivo n, número de etapas de la cadena.
- Un conjunto de entidades Stage, indexadas por $i \in \{1, \dots, n\}$. Cada etapa contiene:
  - $t_i \in \mathbb{Z}_{\geq 0}$: duración normal de la etapa.
  - $c_i \in \mathbb{Z}_{\geq 0}$: costo de acelerar la etapa.
  - $d_i \in \mathbb{Z}_{\geq 0}$: reducción de tiempo al acelerarla.
- Un presupuesto entero $B \in \mathbb{Z}_{\geq 0}$.
- Un tiempo meta $T_{meta} \in \mathbb{Z}_{\geq 0}$.

Los datos deben satisfacer, para cada etapa i,

$$0 \leq d_i \leq t_i.$$

La restricción $d_i \leq t_i$ garantiza que acelerar una etapa no produzca una duración negativa. En la implementación, esta condición se valida durante la construcción de Stage.

MOCHILA 0/1

Si $A \subseteq \{1, \dots, n\}$ denota el subconjunto de etapas aceleradas, entonces:

- costo total de aceleración:

$$C(A) = \sum_{i \in A} c_i ;$$

- reducción total de tiempo:

$$D(A) = \sum_{i \in A} d_i ;$$

- duración normal de la cadena:

$$T_{base} = \sum_{i=1}^{n} t_i ;$$

- duración final de la cadena:

$$T(A) = T_{base} - D(A) = \sum_{i=1}^{n} t_i - \sum_{i \in A} d_i.$$

1.2 Pregunta de decisión

El problema **CHAIN-ACCEL-DEC** pregunta:

¿Existe un subconjunto $A \subseteq \{1, \dots, n\}$ tal que

$$C(A) \leq B$$

y

$$T(A) \leq T_{meta}?$$

La respuesta es **SÍ** si existe dicho subconjunto y **NO** en caso contrario.

La versión de optimización implementada busca maximizar $D(A)$ sujeto a $C(A) \leq B$. Esta versión está directamente relacionada con la versión de decisión: una duración final no mayor que T_{meta} equivale a exigir una reducción mínima

$$D(A) \geq T_{base} - T_{meta}.$$

1.3 Tamaño de la entrada

El tamaño de la entrada se mide en bits de memoria. Sea

$$M_t = \max_i t_i, \qquad M_c = \max_i c_i, \qquad M_d = \max_i d_i.$$

Ignorando factores constantes y la codificación de los identificadores de las etapas, el tamaño es

$$\sum_{i=1}^{n} \lfloor \log_2 t_i \rfloor + \sum_{i=1}^{n} \lfloor \log_2 c_i \rfloor + \sum_{i=1}^{n} \lfloor \log_2 d_i \rfloor + \lfloor \log_2 B \rfloor + \lfloor \log_2 T_{meta} \rfloor.$$

Esta expresión representa la suma de los bits necesarios para codificar los valores numéricos de la instancia; para valores iguales a cero se entiende la convención usual de reservar un bit para su representación. Si se consideran también los bits adicionales de longitud y los factores constantes de la codificación, se obtiene la cota superior asintótica

$$N = O(n\log(M_t) + n\log(M_c) + n\log(M_d) + \log B + \log T_{meta}).$$

Si se omite explícitamente el tamaño de T_{meta} por estar determinado por el resto de la instancia o por considerarse parte del parámetro de consulta, puede expresarse como

$$N = O\left(n\log(\max_i t_i) + n\log(\max_i c_i) + n\log(\max_i d_i) + \log B\right).$$

La distinción entre el valor numérico B y su representación binaria será fundamental para analizar la naturaleza pseudo-polinomial del algoritmo exacto.

---

2. Pertenencia de CHAIN-ACCEL-DEC a NP

Para demostrar que **CHAIN-ACCEL-DEC** pertenece a NP, se proporciona como certificado un subconjunto de etapas

$$A \subseteq \{1, \ldots, n\}$$

que supuestamente puede acelerarse para alcanzar la meta.

2.1 Verificador

El verificador recibe la instancia y el certificado A. Recorre las n etapas una sola vez y calcula:

1. La duración base:

$$T_{base} = \sum_{i=1}^{n} t_i.$$

2. El costo de las etapas incluidas en el certificado:

$$C(A) = \sum_{i \in A} c_i.$$

3. La reducción obtenida:

$$D(A) = \sum_{i \in A} d_i.$$

4. La duración resultante:

$$T(A) = T_{base} - D(A).$$

Finalmente, acepta si y solo si se cumplen simultáneamente:

$$C(A) \leq B \qquad \text{y} \qquad T(A) \leq T_{meta}.$$

En términos de la implementación, estas cantidades corresponden a:

- base_duration para T_{base}.
- total_cost para $C(A)$.
- total_reduction para $D(A)$.
- total_duration para $T(A)$.

Cada etapa se consulta una vez, y las operaciones aritméticas sobre enteros tienen costo polinomial en el tamaño de sus representaciones binarias. Por tanto, el tiempo del verificador es

$$O(n)$$

operaciones aritméticas, o polinomial en N. El certificado tiene longitud polinomial, pues puede representarse mediante un vector booleano de n posiciones. En consecuencia,

CHAIN-ACCEL-DEC ∈ NP.

---

3. Demostración de NP-Dureza mediante reducción desde Mochila 0/1

3.1 Problema origen: KNAPSACK-DEC

Se utiliza la versión de decisión de **Mochila 0/1**, denotada por **KNAPSACK-DEC**.

Una instancia contiene:

- n objetos.
- Para cada objeto i:
  - peso $w_i \in \mathbb{Z}_{\geq 0}$;
  - valor $v_i \in \mathbb{Z}_{\geq 0}$.
- Capacidad de la mochila $W \in \mathbb{Z}_{\geq 0}$.
- Valor meta $V \in \mathbb{Z}_{\geq 0}$.

La pregunta es si existe un subconjunto $S \subseteq \{1, \dots, n\}$ tal que

$$\sum_{i \in S} w_i \leq W$$

y

$$\sum_{i \in S} v_i \geq V.$$

KNAPSACK-DEC es NP-completo. Para probar la NP-dureza de CHAIN-ACCEL-DEC, se transforma cualquier instancia de KNAPSACK-DEC en una instancia equivalente de aceleración.

3.2 Transformación

Dada una instancia

$$(\{(w_i, v_i)\}_{i=1}^{n}, W, V),$$

se construye una cadena con n etapas y se define, para cada i:

$$c_i = w_i, \qquad d_i = v_i, \qquad t_i = v_i.$$

Además, se fijan:

$$B = W$$

y

$$T_{meta} = \sum_{i=1}^{n} v_i - V.$$

La validez del dominio se conserva porque

$$0 \le d_i = v_i = t_i.$$

La transformación copia cada par (w_i, v_i) una vez y calcula una suma de n valores. Por tanto, requiere tiempo $O(n)$ operaciones aritméticas y es polinomial en el tamaño de la instancia codificada.

3.3 Equivalencia bidireccional

Demostraremos que

KNAPSACK-DEC tiene respuesta SÍ ⇔ CHAIN-ACCEL-DEC tiene respuesta SÍ.

Dirección (⇒)

Supóngase que la instancia de KNAPSACK-DEC tiene respuesta SÍ. Entonces existe un subconjunto S tal que

$$\sum_{i \in S} w_i \le W$$

y

MOCHILA 0/1

$$\sum_{i \in S} v_i \geq V.$$

Construimos una solución de cadena acelerando exactamente las etapas de S, es decir, tomamos $A = S$. Por la transformación,

$$C(A) = \sum_{i \in A} c_i = \sum_{i \in S} w_i \leq W = B.$$

Además,

$$T(A) = \sum_{i=1}^{n} t_i - \sum_{i \in A} d_i = \sum_{i=1}^{n} v_i - \sum_{i \in S} v_i \leq \sum_{i=1}^{n} v_i - V = T_{meta}.$$

Por consiguiente, la instancia de CHAIN-ACCEL-DEC también tiene respuesta SÍ.

Dirección (⇐)

Supóngase ahora que la instancia de CHAIN-ACCEL-DEC tiene respuesta SÍ. Entonces existe un subconjunto A tal que

$$\sum_{i \in A} c_i \leq B$$

y

$$\sum_{i=1}^{n} t_i - \sum_{i \in A} d_i \leq T_{meta}.$$

Sustituyendo la transformación, la primera desigualdad queda

$$\sum_{i \in A} w_i \leq W.$$

La segunda desigualdad queda

$$\sum_{i=1}^{n} v_i - \sum_{i \in A} v_i \leq \sum_{i=1}^{n} v_i - V.$$

Al cancelar la suma común y multiplicar por -1, se obtiene

$$\sum_{i \in A} v_i \geq V.$$

Por tanto, el mismo subconjunto A constituye una solución válida para KNAPSACK-DEC. La instancia de mochila tiene respuesta SÍ.

La transformación es polinomial y preserva la respuesta. Como KNAPSACK-DEC es NP-completo,

CHAIN-ACCEL-DEC es NP-difícil.

Combinando esta conclusión con la pertenencia a NP:

CHAIN-ACCEL-DEC es NP-completo.

---

4. NP-Dureza débil, complejidad y arquitectura de programación dinámica

4.1 Recurrencia exacta

La versión de optimización maximiza la reducción de tiempo obtenida con un presupuesto limitado. Sea $DP[i][b]$ la máxima reducción alcanzable usando únicamente las primeras i etapas y un presupuesto disponible de b.

Para la etapa i, existen dos decisiones:

1. **No acelerar la etapa:** se conserva $DP[i-1][b]$.
2. **Acelerar la etapa**, si $c_i \leq b$: se obtiene la reducción d_i además de la mejor solución con presupuesto restante $b - c_i$.

La recurrencia es:

$$DP[i][b] = \max(DP[i-1][b],\ DP[i-1][b-c_i] + d_i) \quad \text{si } c_i \leq b.$$

$$DP[i][b] = DP[i-1][b] \quad \text{si } c_i > b.$$

Las condiciones iniciales son:

$$DP[0][b] = 0 \quad \text{para todo } b \in \{0, \dots, B\},$$

porque sin etapas no puede obtenerse ninguna reducción. No se introduce una condición base independiente para $DP[i][0]$: los estados con presupuesto cero se evalúan directamente mediante la recurrencia. Esto permite tratar de forma natural los casos borde en los que existan etapas con costo nulo ($c_i = 0$), sin imponer una restricción adicional que pueda ocultar ese comportamiento.

El resultado óptimo de reducción es $DP[n][B]$. La duración final óptima es

$$T_{opt} = \sum_{i=1}^{n} t_i - DP[n][B].$$

4.2 Correspondencia con la implementación

La arquitectura de dominio representa el problema mediante los siguientes elementos:

- Stage es la entidad que encapsula name, duration, cost y reduction. Sus invariantes garantizan valores no negativos y $d_i \leq t_i$.
- AccelerationScenario es el objeto que agrupa la tupla de etapas y el presupuesto B.
- OptimizeAcceleration es el caso de uso de aplicación que ejecuta la solución exacta de mochila 0/1.
- DpTraceEntry representa un estado calculado de la tabla, con etapa, índice, presupuesto disponible, reducción y decisión.
- AccelerationResult contiene la duración base, duración final, costo utilizado, reducción total, etapas aceleradas, detalles de las etapas y traza.

En el código, el arreglo

reductions[available_budget]

mantiene la fila actual de valores de DP. Antes de procesar cada etapa se copia la fila anterior mediante previous = reductions[:], evitando que una etapa pueda utilizarse más de una vez. Esta copia implementa la restricción 0/1.

La matriz

choices[stage_index][available_budget]

registra si la solución óptima asociada al estado se obtuvo tomando la etapa. Cuando la alternativa de tomar la etapa mejora la reducción, se almacena True; en caso contrario, se mantiene False, que representa la decisión skip.

La reconstrucción comienza en el estado final (n, B) y recorre las etapas en orden inverso. Si choices[stage_index][remaining_budget] es verdadero, la etapa se incorpora a la solución y se resta su costo del presupuesto restante. Al final se invierte la lista para recuperar el orden original de la cadena.

Cada evaluación también genera un objeto DpTraceEntry con la decisión take o skip. Esta traza proporciona evidencia verificable de los estados evaluados y permite documentar el comportamiento del algoritmo sobre la entrada oficial.

4.3 Deducción de la función de eficiencia

La tabla considera:

- n etapas;
- $B + 1$ valores de presupuesto, desde 0 hasta B.

Por tanto, el número de estados evaluados es

$$n(B + 1) = nB + n = \Theta(nB).$$

Cada estado realiza un número constante de operaciones: consulta de la solución anterior, comparación de alternativas y, cuando corresponde, suma de una reducción. Por ello, el costo por celda es $O(1)$ y el tiempo total es

$$T(n, B) = \Theta(nB).$$

En la implementación, la generación de la traza agrega una operación constante por estado, por lo que conserva la misma cota asintótica:

$$T_{\text{DP+traza}}(n, B) = \Theta(nB).$$

Si solo interesara obtener el valor de la reducción máxima, sin reconstruir el subconjunto de etapas, bastaría un arreglo unidimensional de tamaño $B + 1$, recorrido en orden inverso para preservar la restricción 0/1. En ese caso, el espacio adicional sería

$$S_{valor}(B) = O(B).$$

La implementación del dominio requiere más información: debe reconstruir el subconjunto óptimo mediante la matriz choices y generar una entrada DpTraceEntry para cada estado calculado como evidencia de auditoría. La matriz choices tiene $n(B+1)$ posiciones booleanas y la traza contiene $n(B+1)$ objetos. Por tanto, aunque el vector reductions y la copia previous solo requieren $O(B)$, la reconstrucción y la auditoría justifican teórica y arquitectónicamente un consumo dominante de

$$S_{dominio}(n,B) = \Theta(nB) + \Theta(nB) + O(B) = \Theta(nB).$$

En consecuencia, existe un trade-off explícito: $O(B)$ si solo se necesita el valor óptimo, frente a $\Theta(nB)$ cuando el contrato del dominio exige reconstrucción y trazabilidad completa.

4.4 Comportamiento pseudo-polinomial

La complejidad $O(nB)$ es polinomial en el valor numérico del presupuesto B, pero el tamaño formal de la entrada mide la cantidad de bits necesarios para representar dicho presupuesto.

Sea

$$k = \lceil \log_2(B+1) \rceil$$

el número de bits de la representación binaria de B. Entonces,

$$B < 2^k$$

y, para familias de instancias donde B crece con su representación,

$$B = \Theta(2^k).$$

Al sustituir esta relación en la complejidad del algoritmo se obtiene

$$T(n,B) = O(nB) = O(n2^k).$$

Por tanto, el algoritmo no es necesariamente polinomial en el tamaño binario de la entrada N, pues B puede ser exponencial en el número de bits k. Se denomina **pseudo-polinomial** porque sí es polinomial cuando los valores numéricos de los parámetros, en particular B, se consideran acotados por un polinomio, pero puede ser exponencial respecto de su codificación binaria.

4.5 NP-dureza débil y exactitud

La reducción desde KNAPSACK-DEC demuestra que CHAIN-ACCEL-DEC es NP-difícil. Sin embargo, la reducción utiliza valores numéricos de pesos, costos, valores y presupuesto, y la programación dinámica explota directamente el valor entero de B. Esta dependencia numérica es precisamente la característica de la **NP-dureza débil**.

En particular, no se contradice la NP-dureza que exista un algoritmo exacto $O(nB)$: dicho algoritmo es eficiente para presupuestos pequeños o moderados, pero no garantiza tiempo polinomial en el número de bits de una instancia con un presupuesto numéricamente grande.

La exactitud de la programación dinámica se justifica por inducción sobre i:

- **Caso base:** con cero etapas, la reducción máxima es cero, por lo que $DP[0][b] = 0$.
- **Paso inductivo:** suponiendo correctos los valores para las primeras $i-1$ etapas, cualquier solución con las primeras i etapas pertenece exactamente a una de dos clases: no contiene la etapa i, o contiene la etapa i. La recurrencia toma el máximo entre la mejor solución de la primera clase y la mejor solución de la segunda clase. Por hipótesis inductiva, ambas cantidades son óptimas dentro de su clase; por tanto, su máximo es óptimo para las primeras i etapas.

Al aplicar el paso hasta $i = n$ y $b = B$, se obtiene $DP[n][B]$, la máxima reducción posible bajo el presupuesto. En consecuencia, OptimizeAcceleration devuelve una solución exacta, no aproximada, y la reconstrucción mediante choices recupera un subconjunto de etapas que alcanza esa reducción.

5. Referencias bibliográficas

Bellman, R. (1957). *Dynamic programming*. Princeton University Press.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W. H. Freeman.